

COL331 A2

Sarthak Panda(2022CS51217)

April 2025

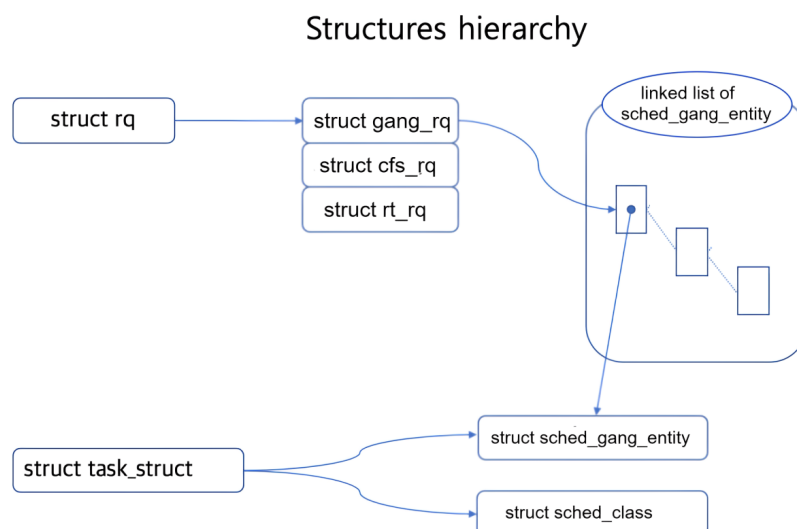
1 Introduction

The current implementation required modification of listed files. Which we added in the patch files and are discussed in following sections.

File	Status
Kbuild	Modified
arch/x86/entry/syscalls/syscall_64.tbl	Modified
hellos/Makefile	New File
hellos/hellos.c	New File
include/asm-generic/vmlinux.lds.h	Modified
include/linux/sched.h	Modified
include/linux/sched/gang.h	New File
include/linux/syscalls.h	Modified
include/uapi/linux/sched.h	Modified
kernel/sched/Makefile	Modified
kernel/sched/core.c	Modified
kernel/sched/gang.c	New File
kernel/sched/sched.h	Modified
tools/include/uapi/linux/sched.h	Modified

2 Implementation of Scheduling Class

2.1 Brief Overview



To design a scheduler we need to have a brief idea of structural hierarchy that they maintain. First of all we must understand each of CPU has it's own runqueue denoted by `rq` in above figure, Which in turn points to scheduler

specific runqueues. These scheduler specific runqueues then enqueue the task that come to that particular cpu with and respective schedling class. In CFS the runqueue is maintained as a rb tree, here in GANG for simplicity we maintain it as a linked list. Further in our implementation each node of linked list is a sched gang entity. Further each task struct has it's unique sched gang entity.

2.2 Data Structure used for Gang run-queue

```

1 struct gang_rq {
2     struct list_head gangs;           // List of all gang entities on this CPU
3     unsigned int nr_running;          // Count of gang threads on this CPU
4     spinlock_t lock;                 // Per-runqueue lock
5 };

```

Listing 1: Kernel code snippet from kernel/sched/sched.h

As discussed above we maintain gang_rq as linked list. Only important thing to note here is that this cpu specific gang_rq must be intialized inside sched_init function located at /kernel/sched/core.c

2.3 Data Structure used for Gang sched-entity

```

1 #ifndef _LINUX_SCHED_GANG_H
2 #define _LINUX_SCHED_GANG_H
3 #include <linux/timer.h>
4 #include <linux/percpu-defs.h>
5 #ifdef __KERNEL__
6     DECLARE_PER_CPU(int, current_gang_id);
7     DECLARE_PER_CPU(struct timer_list, gang_timer);
8 #endif
9 struct gang_thread {
10     int pid;
11     int exec_time;
12     int cpu;
13     int gangid;
14     struct timer_list timer;
15 };
16 #endif

```

Listing 2: Kernel code snippet from include/linux/sched/gang.h

```

1 struct sched_gang_entity {
2     struct gang_thread *gang_thd;
3     struct list_head run_list;
4     struct gang_rq *gang_rq;
5     unsigned int on_rq;
6 };

```

Listing 3: Kernel code snippet from include/linux/sched.h

Usually the sched_entities are initialized in sched_fork callback in core.c , but in our case no such initialization was needed we initialized node on fly during enqueue.

2.4 Implementation of Scheduler Call-Back Functions

There are three essential callback function of a scheduler that must be implemented by a scheduling class for the main scheduler to able to schedule the tasks properly. Below is the code snippet of main scheduler [Taken from book linux kernel development by Robert] Where he describes pick_next_task as scheduler entry point. So to be able to return a valid task_struct upon calling pick_next_task we must first properly enqueue & dequeue task_structs into & out of our scheduler specific runqueue. So these three call backs become essential & fundamental are briefly discussed in below sections.

```

/*
 * Pick up the highest-prio task:
 */
static inline struct task_struct *
pick_next_task(struct rq *rq)
{
    const struct sched_class *class;
    struct task_struct *p;

    /*
     * Optimization: we know that if all tasks are in
     * the fair class we can call that function directly:
     */
    if (likely(rq->nr_running == rq->cfs.nr_running)) {
        p = fair_sched_class.pick_next_task(rq);
        if (likely(p))
            return p;
    }

    class = sched_class_highest;
    for ( ; ; ) {
        p = class->pick_next_task(rq);
        if (p)
            return p;

        /*
         * Will never be NULL as the idle class always
         * returns a non-NULL p:
         */
        class = class->next;
    }
}

```

2.4.1 enqueue_task

As discussed its role is to simply enqueue into our gang run-queue (specific to a cpu).

```

1 static void gang_enqueue_task(struct rq *rq, struct task_struct *p, int flags) {
2     struct gang_rq *grq = &rq->gang;
3     struct sched_gang_entity *gse = &p->gang_se;
4     unsigned long irq_flags;
5     spin_lock_irqsave(&grq->lock, irq_flags);
6     if (!gse->on_rq) {
7         gse->gang_rq = grq;
8         INIT_LIST_HEAD(&gse->run_list);
9         list_add_tail(&gse->run_list, &grq->glists);
10        gse->on_rq = 1;
11        grq->nr_running++;
12    }
13    spin_unlock_irqrestore(&grq->lock, irq_flags);
14 }

```

Listing 4: Kernel code snippet from kernel/sched/gang.c

2.4.2 dequeue_task

```

1 static void gang_dequeue_task(struct rq *rq, struct task_struct *p, int flags) {
2     struct sched_gang_entity *gse = &p->gang_se;
3     struct gang_rq *grq = gse->gang_rq;
4     unsigned long irq_flags;
5     if (!grq || !gse->on_rq)
6         return;
7     spin_lock_irqsave(&grq->lock, irq_flags);
8     if (gse->on_rq) {
9         list_del_init(&gse->run_list);
10        grq->nr_running--;
11        gse->on_rq = 0;
12    }
13    spin_unlock_irqrestore(&grq->lock, irq_flags);
14 }

```

Listing 5: Kernel code snippet from kernel/sched/gang.c

2.4.3 pick_next_task

```
1 static struct task_struct *gang_pick_next_task(struct rq *rq) {
2     struct gang_rq *grq = &rq->gang;
3     struct sched_gang_entity *gse;
4     struct gang_ipi_data *ipi_data;
5     static int last_gang = -1;
6     int current_gang;
7     unsigned long irq_flags;
8     if (rq->cpu == governor_cpu) {
9         struct list_head *pos, *next;
10        bool found = false;
11        //round robin choice of the gangs from the gang run queue
12        spin_lock_irqsave(&grq->lock, irq_flags);
13        list_for_each_safe(pos, next, &grq->gangs) {
14            gse = list_entry(pos, struct sched_gang_entity, run_list);
15            if (gse->gang_thd->gangid > last_gang) {
16                last_gang = gse->gang_thd->gangid;
17                found = true;
18                goto found;
19            }
20        }
21        if (!list_empty(&grq->gangs)) {
22            gse = list_first_entry(&grq->gangs, struct sched_gang_entity, run_list);
23            last_gang = gse->gang_thd->gangid;
24            found = true;
25        }
26    found:
27        if (found) {
28            ipi_data = kmalloc(sizeof(*ipi_data), GFP_ATOMIC);
29            if (!ipi_data) {
30                spin_unlock_irqrestore(&grq->lock, irq_flags);
31                return NULL;
32            }
33            ipi_data->gangid = last_gang;
34            ipi_data->quanta = GANG_QUANTA_MS;
35            //governor writes into his own per cpu region
36            per_cpu(current_gang_id, governor_cpu) = last_gang;
37            //governor sends ipi to write into per cpu region of each online cpu
38            smp_call_function(gang_ipi_handler, ipi_data, 1);
39            kfree(ipi_data);
40        }
41        spin_unlock_irqrestore(&grq->lock, irq_flags);
42    }
43    //each thread reads gang id from the per cpu region.
44    current_gang = per_cpu(current_gang_id, rq->cpu);
45    if (current_gang == -1) return NULL;
46    spin_lock_irqsave(&grq->lock, irq_flags);
47    list_for_each_entry(gse, &grq->gangs, run_list) {
48        if (gse->gang_thd->gangid == current_gang && gse->on_rq) {
49            spin_unlock_irqrestore(&grq->lock, irq_flags);
50            //as soon as they start to work they start their timer of about GANG_QUANTA time.
51            //mod_timer reinitializes and restarts a expired/unused timer
52            mod_timer(&per_cpu(gang_timer, rq->cpu), jiffies_msecs_to_jiffies(GANG_QUANTA_MS));
53            return container_of(gse, struct task_struct, gang_se);
54        }
55    }
56    spin_unlock_irqrestore(&grq->lock, irq_flags);
57    return NULL;
58 }
```

Listing 6: Kernel code snippet from kernel/sched/gang.c

It is call back that forms the main scheduling logic. First of all we maintain two parameters in per cpu region namely `gang_id` and a timer (of type `timer_list`) with a expiration time of 10msec (i.e. `GANG_QUANTA`). The strategy is as gang governor thread is about to get picked up it sends ipi to all of the remaining cpus using `smp_call_function` which basically rewrites the per cpu region variable `gang_id` of all of the cpus to the current `gang_id` of governor (Since governor threads round robin these automatically by this mechanisms) and calls `resched` to reschedule other cpus (so that pick next task gets invoked on other cpus as well). Further then each cpu look up into it's run queue and pick the thread relevant to that `gang_id` in `per_cpu` region (Since there each gang has atmost one thread loaded/pinned to a cpu, this is bound to be unique). Now as soon it finds a valid `task_struct` to be returned each core intialize a timer in `per_cpu` region which on expiration resets the `gang_id` to -1.

```

1 #define GANG_QUANTA_MS 10
2 static int governor_cpu = 0;
3 struct gang_ipi_data {
4     int gangid;
5     int quanta;
6 };
7 DEFINE_PER_CPU(int, current_gang_id);
8 DEFINE_PER_CPU(struct timer_list, gang_timer);
9 static void gang_ipi_handler(void *info){
10     struct gang_ipi_data *data = info;
11     int cpu = smp_processor_id();
12     per_cpu(current_gang_id, cpu) = data->gangid;
13     resched_curr(cpu_rq(cpu));
14 }
15 static void gang_timer_fn(struct timer_list *t)
16 {
17     int cpu = smp_processor_id();
18     per_cpu(current_gang_id, cpu) = -1;
19     resched_curr(cpu_rq(cpu));
20 }
21 static int __init gang_sched_init(void)
22 {
23     int cpu;
24     for_each_possible_cpu(cpu) {
25         timer_setup(&per_cpu(gang_timer, cpu), gang_timer_fn, 0);
26         per_cpu(current_gang_id, cpu) = -1;
27     }
28     return 0;
29 }
30 core_initcall(gang_sched_init);

```

Listing 7: Kernel code snippet from `kernel/sched/gang.c` illustrating per cpu vriable management and intialization

3 Challenges Faced

3.1 Debugging Scheduler Call-Back Function

I had observed that adding `pr_tnk` statements inside these scheduler call back function were leading to a failed boot into the compiled kernel. Most likely the reason was related to deadlocks. I went through some suggestion available online they seem to recommend the usage of dynamic debugging using `pr_debug` and some other seem to suggest `trace_printk`. So debugging the call back functions was a challenging part in this assignment.